

MC 122
Object Oriented Programming (AY Winter 2025-26)

Instructor: Dr. Ankit Vijayvargiya, Faculty Block-4

Course Placement: B.Tech. (MNC), Semester-II

Course Format: 2 Lecture hours and 2 Lab hours.

Course Content: The course begins with a recap of structured programming, revisiting variables, conditionals, loops, arrays, functions, and modular program design. It then introduces the principles of Object-Oriented Programming, covering classes, objects, constructors, inheritance, polymorphism, abstraction, and encapsulation. Students also learn exception handling, file handling, and basic object-oriented design concepts, with laboratory work focused on implementing OOP concepts and developing small applications.

Books:

1. Introducing Python: Modern Computing in Simple Packages by Bill Lubanovic, O'Reilly, 2nd Edition, 2019.
2. Python 3 Object-oriented Programming by Dusty Phillips, Packt Publishing, 2nd Edition, 2015.

Related Resources:

1. Visual Studio Code: <https://code.visualstudio.com/download>
2. Anaconda: <https://www.anaconda.com/download>
3. Python 3.9: <https://www.python.org/downloads/>

Assessment Method: i) Attendance: 10%, ii) Mid-sem Exam-20%, iii) End-sem Exam: 40%, iv) Other components (Project, Viva, Quiz and Assignments): 30%.

Course Outcomes: By the end of the course, students will be able to recall and use structured programming concepts, construct classes and objects to model real-world problems, and apply encapsulation, abstraction, inheritance, and polymorphism effectively in program design. They will also be able to implement robust programs using exception handling, file operations, and modular programming practices. Ultimately, students will gain the ability to analyze problem statements and develop complete object-oriented applications on, integrating appropriate design and coding principles.

Mapping:

PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12	PSO1	PSO2
X	X	X		X				X	X	X	X	X	X

Tentative Lecture Distribution

S. No	Description	No. of Lectures
1	Introduction to Subject , Syllabus, Text and Reference Books Discussion, Objectives, scope, outcomes of the course, and evaluation scheme	01
2	Recap of Structured Programming with Python • Introduction to programming paradigms: procedural vs object-oriented • Variables, data types, operators • Conditional statements and looping constructs, • Data structure: lists, tuples, dictionaries, strings; • Functions: definition, parameters, return values, recursion, modular design Lab Exercise: • Practice basic python syntax and operators • Implement simple programs using conditionals, loops, and functions, • Write recursive functions and modular code	05
3	Fundamentals of Object-Oriented Programming • Introduction to OOP: concepts of classes, objects • Principles: Abstraction, Encapsulation, Inheritance, Polymorphism • Instance variables, instance methods, self • Constructors (<code>__init__</code>) and object lifecycle • Class variables, class methods, static methods Lab Exercise: • Define and create classes with attributes and methods • Implement constructors, instance methods, and class methods • Demonstrate encapsulation using getters and setters	05
4	Inheritance and Polymorphism • Single, multilevel, and multiple inheritance • Method overriding • Use of <code>super()</code> and constructor chaining • Python Method Resolution Order (MRO) • Polymorphism: runtime polymorphism, duck typing • Operator overloading (<code>__add__</code>, <code>__str__</code>, etc.) Lab Exercise: • Implement inheritance hierarchies with method overriding • Demonstrate use of <code>super()</code> • Apply operator overloading for custom classes • Develop small programs illustrating polymorphism	06
5	Exception Handling, File Handling & Modular Programming • Errors vs. exceptions; exception hierarchy in Python • try, except, else, finally blocks • Raising exceptions; creating custom exception classes • File handling: reading/writing text files, JSON, CSV, and binary files • Modules and packages: creating, importing, and using • Overview of Python standard libraries (collections, datetime, itertools)	06

	Lab Exercise: • Implement programs with exception handling • Read/write data to files (text, CSV) • Create and use custom modules and packages • Apply standard library functions to solve programming tasks	
6	Object-Oriented Design • UML class diagrams: classes, relationships (association, aggregation, composition) • Basic object-oriented design principles: modularity, cohesion, low coupling • Introduction to SOLID principles (conceptual overview) • Project design and planning: integrating OOP, file handling, exceptions Lab Exercise: • Design and implement a small application (e.g., student management system, inventory system, library system) • Integrate classes, inheritance, polymorphism, exceptions, and file handling • Prepare UML diagrams and document project design	05
Total Lectures		28

*The extent of topic coverage will be influenced by the collective engagement and participation of the class.